

UNIVERSIDAD NACIONAL DE COLOMBIA

Facultad de Minas

Departamento de Ciencias de la Computación y de la Decisión

Ingeniería de Sistemas e Informática

Programación Orientada a Objetos (2026-2S)

Actividad 2: Desarrollo de Ejercicios OO

Estudiante:
José Alejandro Gómez Berrío
1067902507

Docente:
Walter Hugo Arboleda Mazo

Fecha de Entrega:
8 de Octubre de 2026

1 Repositorio de GitHub

El código fuente compilable en Java y la estructura del proyecto se encuentran alojados en el siguiente repositorio público de GitHub:

https://github.com/alejandrogb09/Actividad2-_POO-_2026-2S

Nota: El archivo README.md del repositorio contiene la información requerida (nombre completo de la universidad, nombre de la actividad, nombre completo del estudiante y nombre completo del docente).

---

2 Ejercicio 2.1 página 63

2.1 Enunciado

Se requiere un programa que modele el concepto de una persona. Una persona posee nombre, apellido, número de documento de identidad y año de nacimiento. La clase debe tener un constructor que inicialice los valores de sus respectivos atributos.

2.2 Diagrama de Clases UML

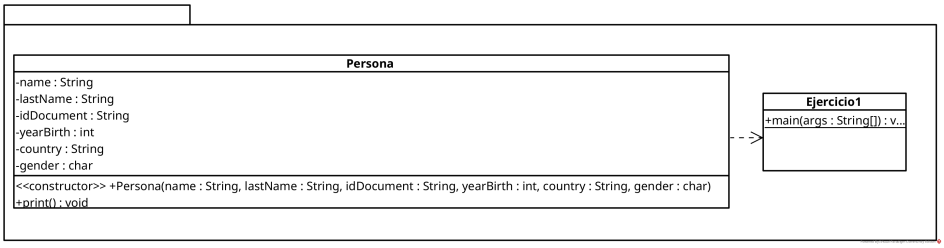


Figure 1: Diagrama de clases UML - Ejercicio 1

2.3 Diagrama de Objetos UML

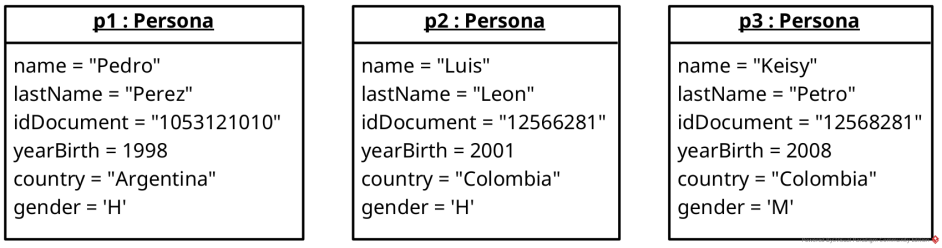


Figure 2: Diagrama de Objetos UML - Ejercicio 1

2.4 Código Fuente en Java

2.4.1 Clase `Persona.java`

```java
package Actividad2;

public class Persona {
    String name;
    String lastName;
    String idDocument;
    int yearBirth;
    String country;
    char gender;

    Persona(String name, String lastName, String idDocument, int yearBirth
   , String country, char gender) {
        this.name = name;
        this.lastName = lastName;
        this.idDocument = idDocument;
        this.yearBirth = yearBirth;
        this.country = country;
        this.gender = gender;
    }

    void print() {
        System.out.println("Name: " + name);
        System.out.println("Last Name: " + lastName);
        System.out.println("Identification Document: " + idDocument);
        System.out.println("Year of Birth: " + yearBirth);
        System.out.println("Country of Birth: " + country);
        System.out.println("Gender: " + gender);
        System.out.println();
    }

}

```

Listing 1: Clase Persona

2.4.2 Clase Principal `Ejercicio1.java`

```java
package Actividad2;

public class Ejercicio1 {
    public static void main(String[] args) {
        Persona p1 = new Persona("Pedro", "Perez", "1053121010", 1998, "
   Argentina", 'H');
        Persona p2 = new Persona("Luis", "Leon", "12566281", 2001, "
   Colombia", 'H');
        Persona p3 = new Persona("Keisy", "Petro", "1064195231", 2008, "
   Colombia", 'M');

        p1.print();
```

```
10          p2.print();
11          p3.print();
12      }
13  }
14
15
```

Listing 2: Clase Principal

3 Ejercicio 2.2 página 66

3.1 Enunciado

Se requiere un programa que modele el concepto de un planeta del sistema solar. Un planeta tiene los siguientes atributos:

- Un nombre de tipo Sring con valor inicial de null.
- Cantidad de satélites de tipo int con valor inicial de cero.
- Masa en kilogramos de tipo double con valor inicial de cero.
- Volumen en kilómetros cúbicos de tipo double con valor inicial de cero.
- Diámetro en kilómetros de tipo int con valor inicial de cero.
- Distancia media al Sol en millones de kilómetros, de tipo int con valor inicial de cero.
- Tipo de planeta de acuerdo con su tamaño, de tipo enumerado con los siguientes valores posibles: GASEOSO, TERRESTRE y ENANO.
- Observable a simple vista, de tipo booleano con valor inicial false.

3.2 Diagrama de Clases UML

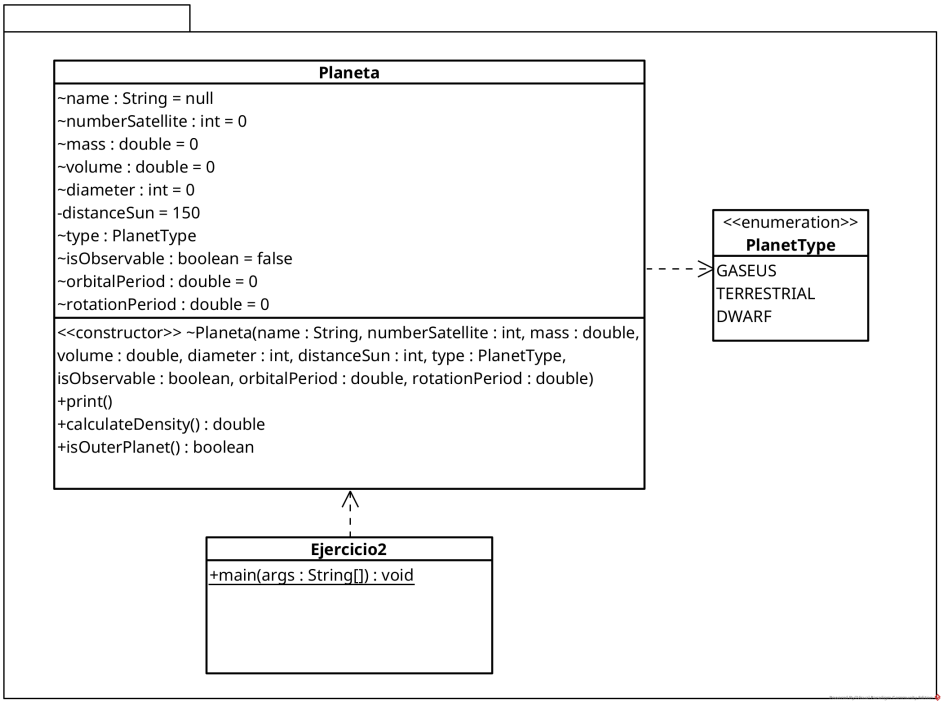


Figure 3: Diagrama de clases UML - Ejercicio 2

3.3 Diagrama de Objetos UML

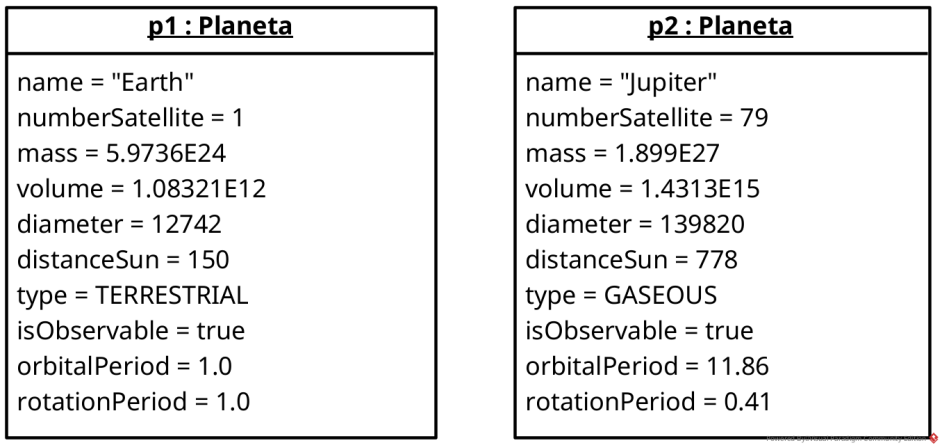


Figure 4: Diagrama de Objetos UML - Ejercicio 2

3.4 Código Fuente en Java

3.4.1 Clase `Planeta.java`

```java
package Actividad2;

public class Planeta {
    String name = null;
    int numberSatellite = 0;
    double mass = 0;
    double volume = 0;
    int diameter = 0;
    int distanceSun = 0;
    enum PlanetType {GASEOUS, TERRESTRIAL, DWARF}
    PlanetType type;
    boolean isObservable = false;
    double orbitalPeriod = 0;
    double rotationPeriod = 0;

    Planeta(String name, int numberSatellite, double mass, double volume,
   int diameter, int distanceSun, PlanetType type, boolean isObservable,
   double orbitalPeriod, double rotationPeriod) {
        this.name = name;
        this.numberSatellite = numberSatellite;
        this.mass = mass;
        this.volume = volume;
        this.diameter = diameter;
        this.distanceSun = distanceSun;
        this.type = type;
        this.isObservable = isObservable;
        this.orbitalPeriod = orbitalPeriod;
        this.rotationPeriod = rotationPeriod;
    }
```

```
28
29      void print() {
30          System.out.println("Name of the planet: " + name);
31          System.out.println("Number of satellites: " + numberSatellite);
32          System.out.println("Mass of the planet: " + mass);
33          System.out.println("Volume of the planet: " + volume);
34          System.out.println("Diameter of the planet: " + diameter);
35          System.out.println("Distance from the sun (M km): " + distanceSun)
    ;
36          System.out.println("Type of planet: " + type);
37          System.out.println("Is it observable: " + isObservable);
38          System.out.println("The orbital period is (years): " +
    orbitalPeriod);
39          System.out.println("The rotation period is (days): " +
    rotationPeriod);
40      }
41
42      double calculateDensity() {
43          return mass / volume;
44      }
45
46      boolean isOuterPlanet() {
47          double distanceInUA = this.distanceSun / 149.597870;
48
49          return distanceInUA > 3.4;
50      }
51  }
52
```

Listing 3: Clase Planeta

3.4.2 Clase Principal `Ejercicio2.java`

```
1  package Actividad2;
2
3  public class Ejercicio2 {
4      public static void main(String[] args) {
5          Planeta p1 = new Planeta("Earth", 1, 5.9736E24, 1.08321E12, 12742,
       150, Planeta.PlanetType.TERRESTRIAL, true, 1.0, 1.0);
6          p1.print();
7          System.out.println("Density of the planet = " + p1.
    calculateDensity());
8          System.out.println("Is it an outer planet = " + p1.isOuterPlanet()
    );
9          System.out.println();
10
11         Planeta p2 = new Planeta("Jupiter", 79, 1.899E27, 1.4313E15,
    139820, 778, Planeta.PlanetType.GASEOUS, true, 11.86, 0.41);
12         p2.print();
13         System.out.println("Density of the planet = " + p2.
    calculateDensity());
14         System.out.println("Is it an outer planet = " + p2.isOuterPlanet()
    );
```

```
15          System.out.println();
16      }
17  }
18
```

Listing 4: Clase Principal

4 Ejercicio 2.3 página 66

4.1 Enunciado

Se requiere un programa que modele el concepto de un automóvil. Un automóvil tiene los siguientes atributos:

- Marca: el nombre del fabricante.
- Modelo: año de fabricación.
- Motor: volumen en litros del cilindraje del motor de un automóvil.
- Tipo de combustible: valor enumerado con los posibles valores de gasolina, bioetanol, diésel, biodiésel, gas natural.
- Tipo de automóvil: valor enumerado con los posibles valores de carro de ciudad, subcompacto, compacto, familiar, ejecutivo, SUV.
- Número de puertas: cantidad de puertas.
- Cantidad de asientos: número de asientos disponibles que tiene el vehículo.
- Velocidad máxima: velocidad máxima sostenida por el vehículo en km/h.
- Color: valor enumerado con los posibles valores de blanco, negro, rojo, naranja, amarillo, verde, azul, violeta.
- Velocidad actual: velocidad del vehículo en un momento dado.

4.2 Diagrama de Clases UML

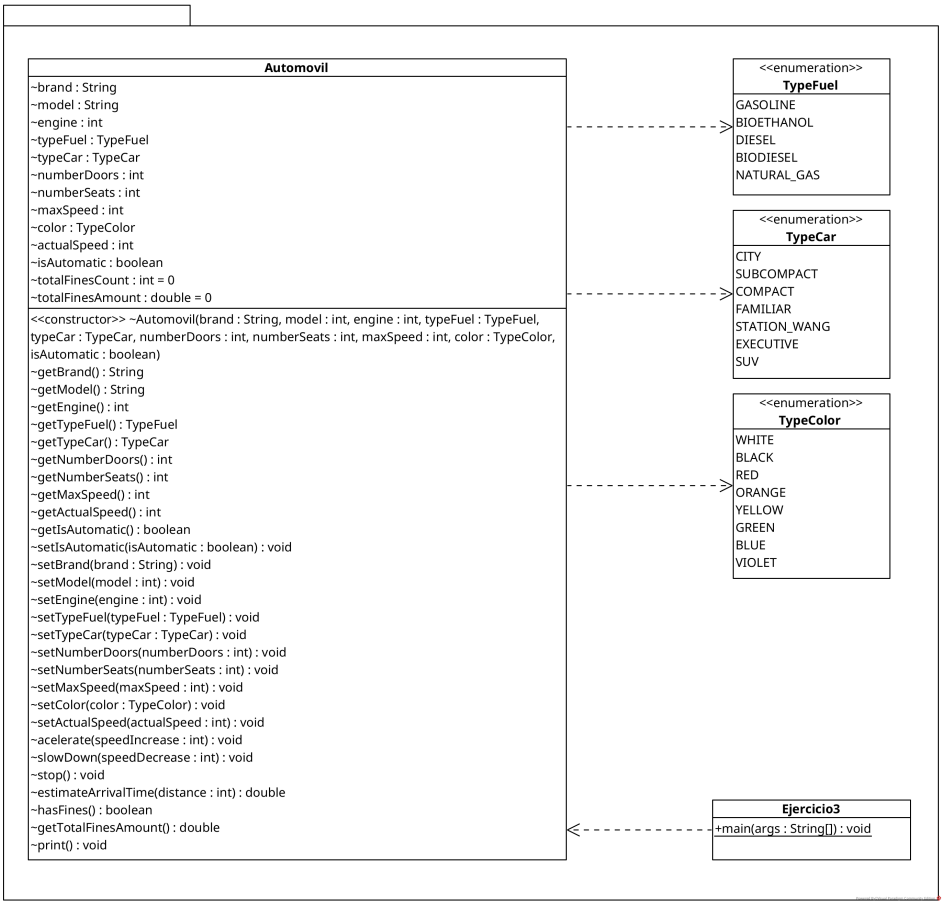


Figure 5: Diagrama de clases UML - Ejercicio 3

4.3 Diagrama de Objetos UML

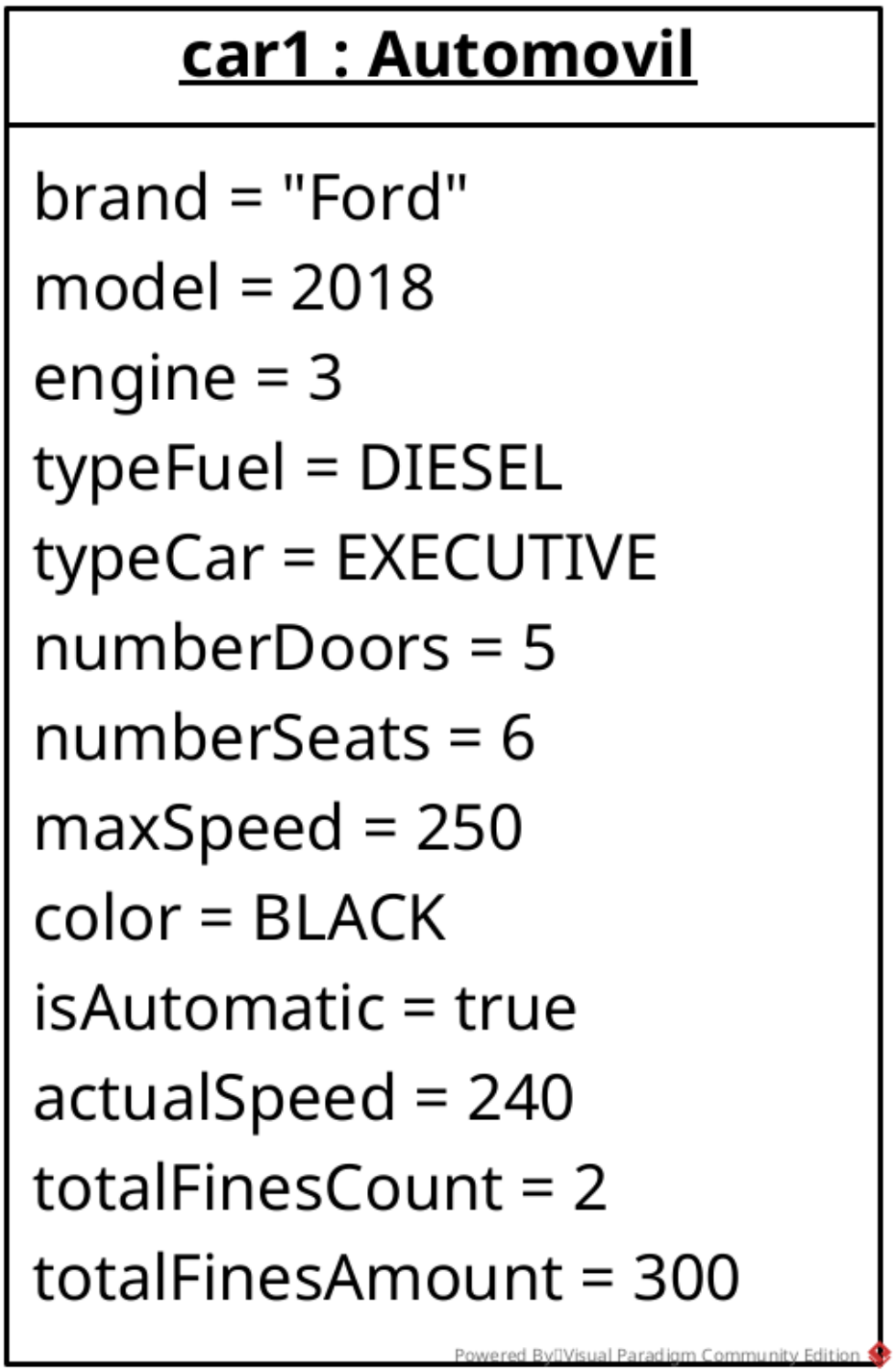


Figure 6: Diagrama de Objetos UML - Ejercicio 3

4.4 Código Fuente en Java

4.4.1 Clase `Automovil.java`

```java
package Actividad2;

public class Automovil {
    String brand;
    int model;
    int engine;
    enum TypeFuel{GASOLINE,BIOETHANOL, DIESEL, BIODIESEL, NATURAL_GAS}
    TypeFuel typeFuel;
    enum TypeCar{CITY, SUBCOMPACT, COMPACT, FAMILIAR, STATION_WANG,
   EXECUTIVE, SUV}
    TypeCar typeCar;
    int numberDoors;
    int numberSeats;
    int maxSpeed;
    enum TypeColor{WHITE, BLACK, RED, ORANGE, YELLOW, GREEN, BLUE, VIOLET}
    TypeColor color;
    int actualSpeed;
    boolean isAutomatic;

    int totalFinesCount = 0;
    double totalFinesAmount = 0;

    Automovil(String brand, int model, int engine, TypeFuel typeFuel,
   TypeCar typeCar, int numberDoors, int numberSeats, int maxSpeed,
   TypeColor color, boolean isAutomatic) {
        this.brand = brand;
        this.model = model;
        this.engine = engine;
        this.typeFuel = typeFuel;
        this.typeCar = typeCar;
        this.numberDoors = numberDoors;
        this.numberSeats = numberSeats;
        this.maxSpeed = maxSpeed;
        this.color = color;
        this.isAutomatic = isAutomatic;
    }

    String getBrand() {
        return brand;
    }

    int getModel() {
        return model;
    }

    int getEngine() {
        return engine;
    }

    TypeFuel getTypeFuel() {
```

```
48          return typeFuel;
49      }
50
51      TypeCar getTypeCar() {
52          return typeCar;
53      }
54
55      int getNumberDoors() {
56          return numberDoors;
57      }
58
59      int getNumberSeats() {
60          return numberSeats;
61      }
62
63      int getMaxSpeed() {
64          return maxSpeed;
65      }
66
67      int getActualSpeed() {
68          return actualSpeed;
69      }
70
71      boolean getIsAutomatic() {
72          return isAutomatic;
73      }
74
75      void setIsAutomatic(boolean isAutomatic) {
76          this.isAutomatic = isAutomatic;
77      }
78
79      void setBrand(String brand) {
80          this.brand = brand;
81      }
82
83      void setModel(int model) {
84          this.model = model;
85      }
86
87      void setEngine(int engine) {
88          this.engine = engine;
89      }
90
91      void setTypeFuel(TypeFuel typeFuel) {
92          this.typeFuel = typeFuel;
93      }
94
95      void setTypeCar(TypeCar typeCar) {
96          this.typeCar = typeCar;
97      }
98
99      void setNumberDoors(int numberDoors) {
100         this.numberDoors = numberDoors;
101     }
```

```
102
103     void setNumberSeats(int numberSeats) {
104         this.numberSeats = numberSeats;
105     }
106
107     void setMaxSpeed(int maxSpeed) {
108         this.maxSpeed = maxSpeed;
109     }
110
111     void setColor(TypeColor color) {
112         this.color = color;
113     }
114
115     void setActualSpeed(int actualSpeed) {
116         this.actualSpeed = actualSpeed;
117     }
118
119     void acelerate(int speedIncrease) {
120         if ((actualSpeed + speedIncrease) <= maxSpeed) {
121             actualSpeed = actualSpeed + speedIncrease;
122         } else {
123             System.out.println("You cannot accelerate at a speed greater
    than the car's maximum speed. Fine generated!");
124             totalFinesCount++;
125             totalFinesAmount += 150.0;
126         }
127     }
128
129     void slowDown(int speedDecrease) {
130         if ((actualSpeed - speedDecrease) > 0) {
131             actualSpeed  = actualSpeed - speedDecrease;
132         } else {
133             System.out.println("You cannot decrement at a negative rate.")
    ;
134         }
135     }
136
137     void stop() {
138         actualSpeed = 0;
139     }
140
141     double estimateArrivalTime(int distance) {
142         return distance / actualSpeed;
143     }
144
145     boolean hasFines() {
146         return totalFinesCount > 0;
147     }
148
149     double getTotalFinesAmount() {
150         return totalFinesAmount;
151     }
152
153     void print() {
```

```
        System.out.println("Brand: " + brand);
        System.out.println("Model: " + model);
        System.out.println("Engine: " + engine);
        System.out.println("Type of Fuel: " + typeFuel);
        System.out.println("Type of Car: " + typeCar);
        System.out.println("Number of Doors: " + numberDoors);
        System.out.println("Number of Seats: " + numberSeats);
        System.out.println("Max Speed: " + maxSpeed);
        System.out.println("Color: " + color);
        System.out.println("Is Automatic: " + isAutomatic);
    }
}
```

Listing 5: Clase Automovil

4.4.2 Clase Principal `Ejercicio3.java`

```
package Actividad2;

public class Ejercicio3 {
    public static void main(String[] args) {
        Automovil car1 = new Automovil("Ford", 2018, 3, Automovil.TypeFuel
    .DIESEL, Automovil.TypeCar.EXECUTIVE, 5, 6, 250, Automovil.TypeColor.
    BLACK, true);

        car1.print();
        car1.setActualSpeed(240);
        System.out.println("Actual Speed: " + car1.actualSpeed);

        car1.acelerate(20);

        car1.acelerate(15);

        System.out.println("Does the car have fines? " + car1.hasFines());
        System.out.println("Total value of fines: $" + car1.
    getTotalFinesAmount());
    }
}
```

Listing 6: Clase Principal

5 Ejercicio 2.4 página 86

5.1 Enunciado

Se requiere un programa que modele varias figuras geométricas: el círculo, el rectángulo, el cuadrado y el triángulo rectángulo.

5.2 Diagrama de Clases UML

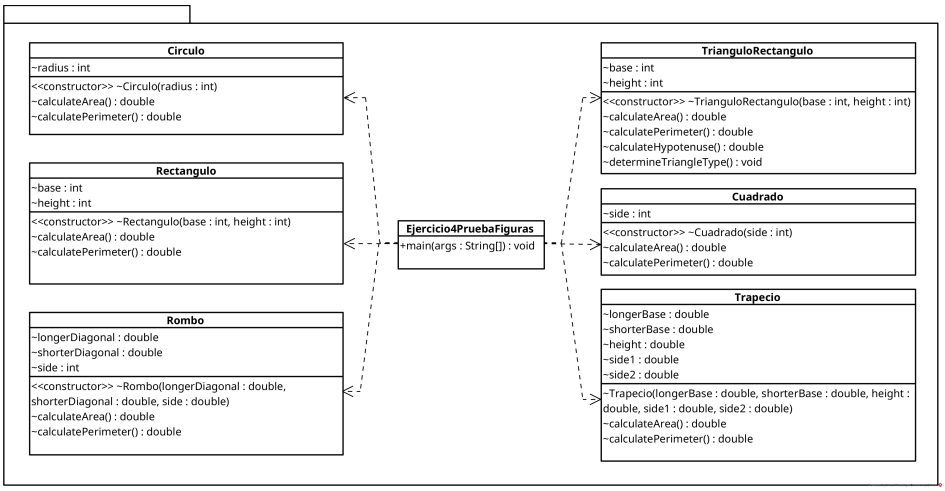


Figure 7: Diagrama de clases UML - Ejercicio 4

5.3 Diagrama de Objetos UML

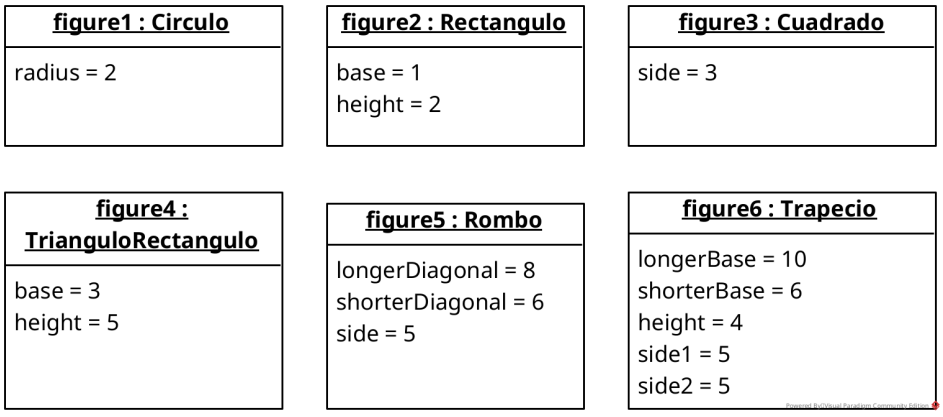


Figure 8: Diagrama de Objetos UML - Ejercicio 4

5.4 Código Fuente en Java

5.4.1 Clase `Circulo.java`

```java
package Actividad2;

public class Circulo {
    int radius;

    Circulo(int radius) {
        this.radius = radius;
    }

    double calculateArea() {
        return Math.PI * Math.pow(radius, 2);
    }

    double calculatePerimeter() {
        return 2 * Math.PI * radius;
    }
}

```

Listing 7: Clase Circulo

5.4.2 Clase `Rectangulo.java`

```java
package Actividad2;

public class Rectangulo {
    int base;
    int height;

    Rectangulo(int base, int height) {
        this.base = base;
        this.height = height;
    }

    double calculateArea() {
        return base * height;
    }

    double calculatePerimeter() {
        return (2 * base) + (2 * height);
    }
}

```

Listing 8: Clase Rectangulo

5.4.3 Clase `Cuadrado.java`

```java
package Actividad2;

public class Cuadrado {
```

```
    int side;

    public Cuadrado(int side) {
        this.side = side;
    }

    double calculateArea() {
        return Math.pow(side, 2);
    }

    double calculatePerimeter() {
        return 4 * side;
    }
}


```

Listing 9: Clase Cuadrado

5.4.4 Clase `Rombo.java`

```
package Actividad2;

public class Rombo {
    double longerDiagonal;
    double shorterDiagonal;
    double side;

    public Rombo(double longerDiagonal, double shorterDiagonal, double
    side) {
        this.longerDiagonal = longerDiagonal;
        this.shorterDiagonal = shorterDiagonal;
        this.side = side;
    }

    double calculateArea() {
        return (longerDiagonal * shorterDiagonal) / 2;
    }

    double calculatePerimeter() {
        return 4 * side;
    }
}

```

Listing 10: Clase Rombo

5.4.5 Clase `Trapecio.java`

```
package Actividad2;

public class Trapecio {
```

```
    double longerBase;
    double shorterBase;
    double height;
    double side1;
    double side2;

    public Trapecio(double longerBase, double shorterBase, double height,
   double side1, double side2) {
        this.longerBase = longerBase;
        this.shorterBase = shorterBase;
        this.height = height;
        this.side1 = side1;
        this.side2 = side2;
    }

    double calculateArea() {
        return ((longerBase + shorterBase) * height) / 2;
    }

    double calculatePerimeter() {
        return longerBase + shorterBase + side1 + side2;
    }
}


```

Listing 11: Clase Trapecio

5.4.6 Clase TrianguloRectangulo.java

```
package Actividad2;

public class TrianguloRectangulo {
    int base;
    int height;

    public TrianguloRectangulo(int base, int height) {
        this.base = base;
        this.height = height;
    }

    double calculateArea() {
        return (base * height) / 2;
    }

    double calculatePerimeter() {
        return (base + height + calculateHypotenuse());
    }

    double calculateHypotenuse() {
        return Math.pow((Math.pow(base, 2) + Math.pow(height, 2)), 0.5);
    }

```

```
    void determineTrianguleType() {
        if ((base == height) && (base == calculateHypotenuse()) && (height == calculateHypotenuse())) {
            System.out.println("It is an equilateral triangle");
        } else if ((base != height) && (base != calculateHypotenuse()) && (height != calculateHypotenuse())) {
            System.out.println("It is a scalene triangle");
        } else {
            System.out.println("It is an isosceles triangle");
        }
    }
}


```

Listing 12: Clase Triangulo Rectangulo

5.4.7 Clase Principal `Ejercicio4PruebaFiguras.java`

```
package Actividad2;

public class Ejercicio4PruebaFiguras {
    public static void main(String[] args) {
        Circulo figure1 = new Circulo(2);
        Rectangulo figure2 = new Rectangulo(1, 2);
        Cuadrado figure3 = new Cuadrado(3);
        TrianguloRectangulo figure4 = new TrianguloRectangulo(3, 5);
        Rombo figure5 = new Rombo(8, 6, 5);
        Trapecio figure6 = new Trapecio(10, 6, 4, 5, 5);

        System.out.println("The area of the circle is: " + figure1.calculateArea());
        System.out.println("The perimeter of the circle is: " + figure1.calculatePerimeter());
        System.out.println();
        System.out.println("The area of the rectangle is: " + figure2.calculateArea());
        System.out.println("The perimeter of the rectangle is: " + figure2.calculatePerimeter());
        System.out.println();
        System.out.println("The area of the square is: " + figure3.calculateArea());
        System.out.println("The perimeter of the square is: " + figure3.calculatePerimeter());
        System.out.println();
        System.out.println("The area of the triangle is: " + figure4.calculateArea());
        System.out.println("The perimeter of the triangle is: " + figure4.calculatePerimeter());
        figure4.determineTrianguleType();
        System.out.println();
        System.out.println("The area of the rhombus is: " + figure5.calculateArea());
```

```
26          System.out.println("The perimeter of the rhombus is: " + figure5.
    calculatePerimeter());
27          System.out.println();
28          System.out.println("The area of the trapezoid is: " + figure6.
    calculateArea());
29          System.out.println("The perimeter of the trapezoid is: " + figure6
    .calculatePerimeter());
30      }
31  }
32
```

Listing 13: Clase Principal

6 Ejercicio 2.5 página 95

6.1 Enunciado

Se requiere un programa que modele una cuenta bancaria.

6.2 Diagrama de Clases UML

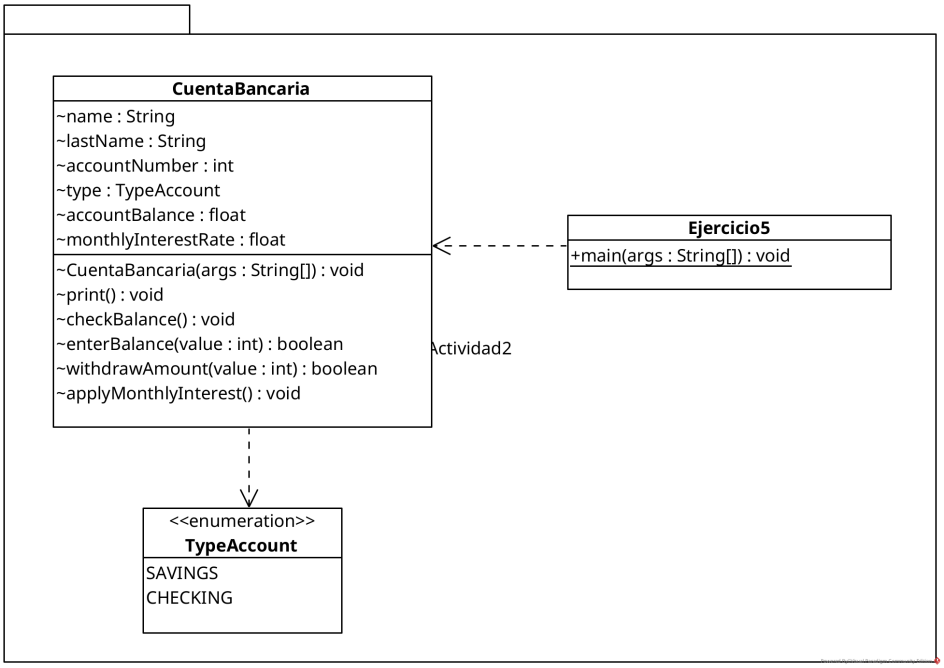


Figure 9: Diagrama de clases UML - Ejercicio 5

6.3 Diagrama de Objetos UML

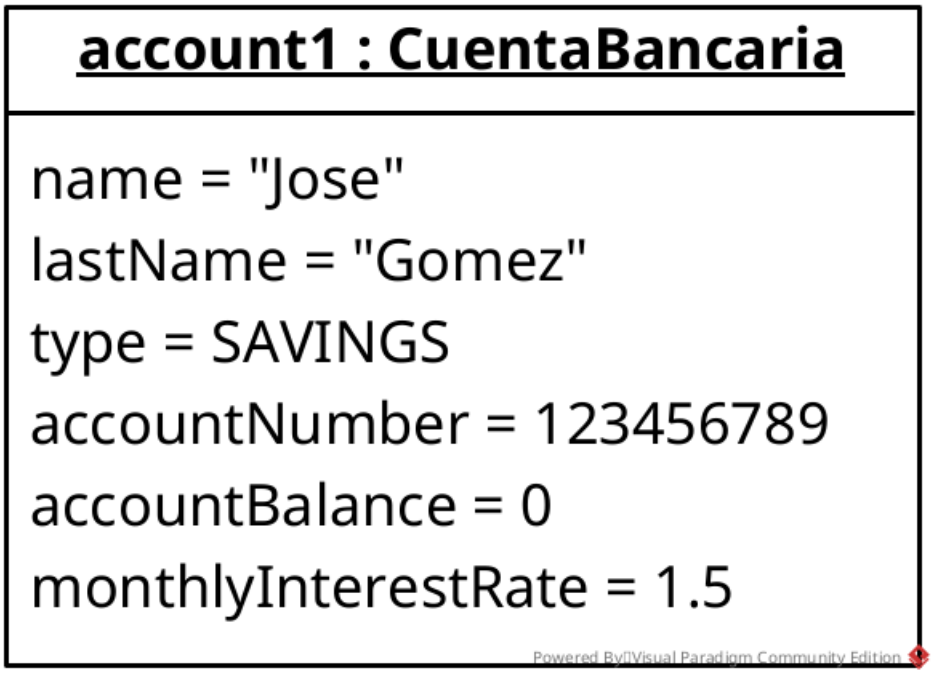


Figure 10: Diagrama de Objetos UML - Ejercicio 5

6.4 Código Fuente en Java

6.4.1 Clase `CuentaBancaria.java`

```
1  package Actividad2;
2
3  public class CuentaBancaria {
4      String name;
5      String lastName;
6      int accountNumber;
7      enum TypeAccount {SAVINGS, CHECKING};
8      TypeAccount type;
9      float accountBalance = 0;
10     float monthlyInterestRate;
11
12     CuentaBancaria(String name, String lastName, int accountNumber,
       TypeAccount type, float monthlyInterestRate) {
13         this.name = name;
14         this.lastName = lastName;
15         this.accountNumber = accountNumber;
16         this.type = type;
17         this.monthlyInterestRate = monthlyInterestRate;
18     }
19
20     void print() {
21         System.out.println("Name: " + name);
```

```
22          System.out.println("Last name: " + lastName);
23          System.out.println("Bank account number: " + accountNumber);
24          System.out.println("Account balance: $" + accountBalance);
25          System.out.println("Monthly interest rate: " + monthlyInterestRate
        + "%");
26      }
27
28      void checkBalance() {
29          System.out.println("The actual balance is: $" + accountBalance);
30      }
31
32      boolean enterBalance(int value) {
33          if (value > 0) {
34              accountBalance = accountBalance + value;
35              System.out.println("$" + value + " has been deposited into the
        account. The new balance is $" + accountBalance);
36              return true;
37          } else {
38              System.out.println("The value to be entered must be greater
       than zero.");
39              return false;
40          }
41      }
42
43      boolean withdrawAmount(int value) {
44          if ((value > 0) && (value <= accountBalance)) {
45              accountBalance = accountBalance - value;
46              System.out.println("$" + value + " has been withdrawn from the
        account. The new balance is $" + accountBalance);
47              return true;
48          } else {
49              System.out.println("The amount to be withdrawn must be less
       than the current balance.");
50              return false;
51          }
52      }
53
54      void applyMonthlyInterest() {
55          float interestAmount = accountBalance * (monthlyInterestRate /
       100);
56          accountBalance = accountBalance + interestAmount;
57          System.out.println("Interest applied (" + monthlyInterestRate + "
       %). The new balance is $" + accountBalance);
58      }
59  }
60
```

Listing 14: Clase Cuenta Bancaria

6.4.2 Clase Principal `Ejercicio5.java`

```
1  package Actividad2;
2
```

```java
public class Ejercicio5 {
    public static void main(String[] args) {
        CuentaBancaria account1 = new CuentaBancaria("Jose", "Gomez",
    123456789, CuentaBancaria.TypeAccount.CHECKING, 1.5f);

        account1.print();
        account1.enterBalance(200000);
        account1.enterBalance(300000);
        account1.withdrawAmount(400000);
        account1.applyMonthlyInterest();
    }
}

```

Listing 15: Clase Principal